Supervised Learning for Binary Intrusion Detection on WUSTL-IIoT-2021

By

Fahmida Faiza Ananna

Technical Coursework Report



School of Computing and Creative Technologies

Under the Faculty of Environment and Technology

UWE BRISTOL

A coursework report submitted to the University of the West of England in accordance with the requirements of the School of Computing and Creative Technologies, Faculty of Environment and Technology.

30 April 2026

Declaration of own work

I declare that the work in this coursework report was carried out in accordance with the requirements of the University's Regulations and Code of Practice and that it has not been submitted for any other academic award. Except where indicated by specific reference in the text, the work is the candidate's own work. Work done in collaboration with, or with the assistance of, others, is indicated as such. Any views expressed in the report are those of the author.

Fahmida Faiza Ananna, 30 April 2026

Acknowledgement

I would like to thank the module team for the guidance provided through the coursework requirements and assessment feedback.

Abstract

Industrial Internet of Things (IIoT) systems require intrusion detection methods that remain reliable under strong class imbalance. This report evaluates supervised binary classification on WUSTL-IIoT-2021, where each network flow is labelled as normal or attack. The dataset contains 1,107,448 normal flows and 87,016 attack flows; within the attack class, DoS accounts for 78,305 flows, while CommInj and Backdoor contain only 259 and 212 flows respectively. The analysis therefore uses leakage-aware preprocessing, an 80/20 stratified split, univariate, bivariate and multivariate evidence, and a comparative evaluation of supervised classifiers. Since attack traffic represents only 7.285% of the dataset, PR-AUC, recall, F1 and MCC are emphasised over accuracy alone. The tuned Random Forest achieves the strongest final profile, with PR-AUC 0.999998 and a confusion matrix of TN=221,482, FP=8, FN=5 and TP=17,398. Its remaining missed attacks are concentrated in Backdoor and CommInj, so the final recommendation is Random Forest with rare-family monitoring rather than a claim of uniform attack-family performance.

Number of words in the report: approximately 1,986 words, excluding tables, figures, captions, references and front matter.

Contents

Page

1 Introduction **1**
1.1 Motivation and problem setting . . . 1
1.2 Objectives and evaluation focus . . . 1
1.2.1 Study workflow overview . . . 2

2 Literature Review **2**
2.1 Literature overview . . . 2
2.1.1 IIoT intrusion detection and datasets . . . 2
2.1.2 Evaluation metrics under imbalance . . . 3
2.1.3 Classifier families and comparative design . . . 3

3 Research Methodology **3**
3.1 Benchmark, target and split protocol . . . 3
3.1.1 Dataset summary . . . 4
3.1.2 Leakage control and retained modelling view . . . 5
3.2 Supervised models and pipelines . . . 6
3.2.1 Algorithm design rationale . . . 6

4 Results **7**
4.1 Univariate evidence . . . 7
4.1.1 Class and attack-family distributions . . . 7
4.1.2 Data-quality summary . . . 8
4.2 Bivariate analysis . . . 9
4.2.1 Feature-target effects . . . 9
4.2.2 Highly correlated feature pairs . . . 10
4.3 Multivariate evidence . . . 10
4.3.1 PCA projection . . . 11
4.3.2 Correlation heatmap . . . 12
4.4 Comparative model performance . . . 12
4.4.1 Held-out metric comparison . . . 13
4.4.2 Precision–recall and ROC curves . . . 14
4.4.3 Confusion matrix for the final model . . . 14
4.5 Runtime and operational diagnostics . . . 15
4.5.1 Training time, prediction time and footprint . . . 15
4.5.2 Attack-family recall and prediction latency . . . 16
4.6 Final model selection . . . 17
4.6.1 Decision matrix and summary . . . 17

5 Discussion and Conclusion **18**
5.1 Synthesis of findings . . . 18
5.2 Rejected models and deployment trade-offs . . . 18
5.3 Limitations and final recommendation . . . 18

A Supplementary Reproducibility Note **19**
A.1 Reproducibility materials . . . 19

References **21**

List of Tables

3.1 Dataset summary 4
3.2 Leakage control 5
3.3 Algorithm design 6

4.1 Class and attack-family counts 8
4.2 Data-quality checks 8
4.3 Bivariate feature–target effects 9
4.4 Correlated pairs 10
4.5 Model comparison 13
4.6 Runtime and suitability 15
4.7 Decision matrix 17

List of Figures

1.1 Evaluation workflow 2

3.1 Pipeline architecture 5

4.1 Class and attack-family skew 7
4.2 PCA projection 11
4.3 Correlation heatmap 12
4.4 PR and ROC curves 14
4.5 Random Forest confusion matrix 14
4.6 Operational diagnostics 16

1 Introduction

1.1 Motivation and problem setting

IIoT environments connect operational technology, sensors, controllers and monitoring services, increasing both visibility and cyber-attack exposure Rathee et al., 2024. Intrusion detection is therefore a practical supervised classification problem: each network flow must be assigned to normal or attack while controlling both false alarms and missed attacks. The operational cost is asymmetric, because too many false alerts reduce trust in the detector, while missed attacks can hide unsafe or disruptive activity.

1.2 Objectives and evaluation focus

Accuracy is insufficient because attacks are rare. The report therefore prioritises PR-AUC, recall, F1, MCC and confusion-matrix errors Chawla et al., 2002; Davis and Goadrich, 2006; Cullerne Bown, 2024; Cruz Huayanay, Bazán and Russo, 2024. The objectives are to build a leakage-aware feature view, use EDA to explain the modelling problem, compare supervised classifiers under one protocol, and connect the final decision to runtime and rare-family behaviour. This keeps the argument tied to evidence: each plot or table is used either to justify a modelling choice or to expose a remaining weakness.

The report therefore avoids a single headline score and instead checks whether the same conclusion is supported by distributions, correlations, aggregate metrics, error counts, family recall and deployment cost before selecting a detector.

1.2.1 Study workflow overview

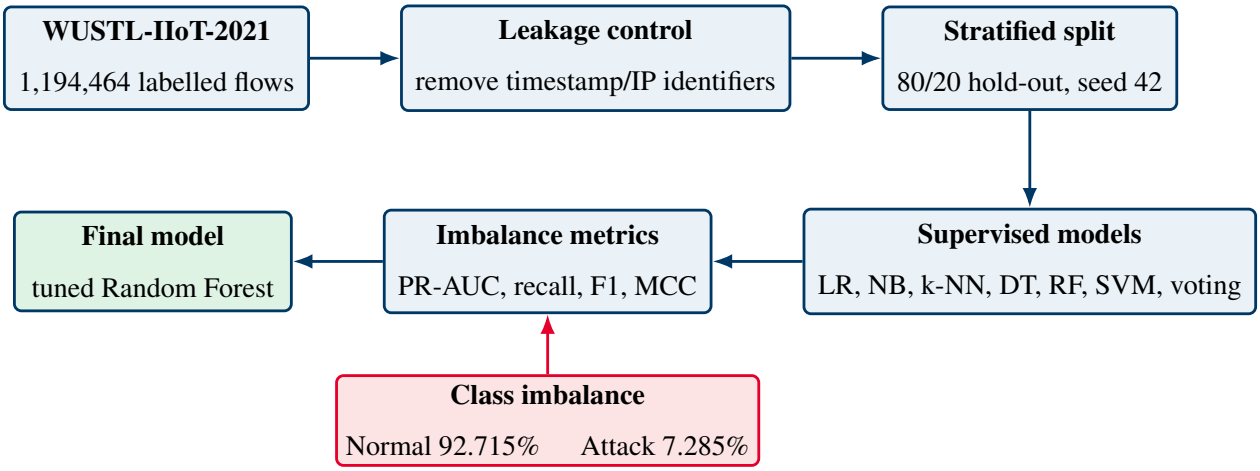


Figure 1.1: Evaluation workflow and class-imbalance context.

Figure 1.1 fixes the experimental order: leakage control, a shared 80/20 split, supervised training, imbalance-aware metrics and final Random Forest selection. The 7.285% attack rate explains why PR-AUC, recall, F1 and MCC drive the later decision.

2 Literature Review

2.1 Literature overview

This chapter links the experiment to IIoT intrusion detection, imbalanced evaluation and comparative classifier design. The review is deliberately narrow because the report is experimental: the literature is used to justify the benchmark choice, the metrics and the set of candidate classifiers.

2.1.1 IIoT intrusion detection and datasets

IIoT intrusion detection is important because industrial networks join cyber and physical processes. Recent studies show that machine learning can detect abnormal flow behaviour, but results depend on preprocessing, class imbalance and dataset choice Eid et al., 2024; Ismail, Dandan and Qushou, 2025. WUSTL-IIoT-2021 matches these concerns because it is large, tabular and strongly imbal-

anced. It is therefore suitable for testing whether high reported performance remains meaningful when rare attacks and possible leakage are examined.

2.1.2 Evaluation metrics under imbalance

ROC-AUC is useful for ranking, but precision-recall analysis is more informative when attacks are rare Davis and Goadrich, 2006; Fawcett, 2006; Chen et al., 2024. Precision measures alert quality, recall measures missed attacks, F1 balances both and MCC remains stable under imbalance Powers, 2011; Sokolova and Lapalme, 2009. PR-AUC and confusion-matrix behaviour are therefore primary evidence, with ROC-AUC kept as a supporting ranking metric rather than the final decision criterion.

2.1.3 Classifier families and comparative design

The algorithms test different assumptions. Logistic Regression and Linear SVM test linear separation Cortes and Vapnik, 1995. Gaussian NB is fast but assumes conditional independence Bishop, 2006. k-NN uses local neighbourhoods but can be slow at prediction time. Decision Tree and Random Forest test nonlinear tree splits, with Random Forest reducing single-tree instability through bagging Breiman, 2001; Hastie, Tibshirani and Friedman, 2009. These contrasts make the later rejection decisions traceable instead of arbitrary.

3 Research Methodology

3.1 Benchmark, target and split protocol

WUSTL-IIoT-2021 contains 1,194,464 labelled flows and 49 raw attributes. The target is binary: normal or attack. `Traffic` is excluded from training and retained only for post-hoc attack-family analysis. Timestamp, address and flow-identifier fields are removed to reduce leakage, leaving 41 numeric predictors. All models use the same stratified 80/20 split with seed 42, preserving the

7.285% attack rate in the test fold. This ensures that differences between models come from the algorithms and pipelines rather than from different splits.

3.1.1 Dataset summary

Property	Value
Dataset	WUSTL-IIoT-2021
Total flows	1,194,464
Raw columns	49
Normal flows	1,107,448
Attack flows	87,016
Attack rate	7.285%
Predictive features	41
Split	Stratified 80/20, seed 42

Table 3.1: Dataset summary and modelling formulation.

Table 3.1 fixes the scale for all later results: only 87,016 of 1,194,464 flows are attacks, roughly one in fourteen records. The stratified split keeps this 7.285% prevalence in the held-out fold, so recall and PR-AUC are interpreted under the same class balance seen in the full dataset.

3.1.2 Leakage control and retained modelling view

Field group	Columns	Decision
Timestamps	`StartTime`, `LastTime`	Removed to avoid temporal leakage.
Network identifiers	`SrcAddr`, `DstAddr`	Removed to reduce identity memorisation.
IP identifiers	`sIpId`, `dIpId`	Removed as non-generalisable identifiers.
Metadata	`Traffic`	Kept only for attack-family analysis.
Prediction label	`Target`	Used as binary target.

Table 3.2: Leakage-control and retained modelling view.

Table 3.2 explains why only 41 predictors are used. Timestamp and address fields could encourage memorisation, while `Traffic` is held back so the later family plot remains an error analysis, not a training label.

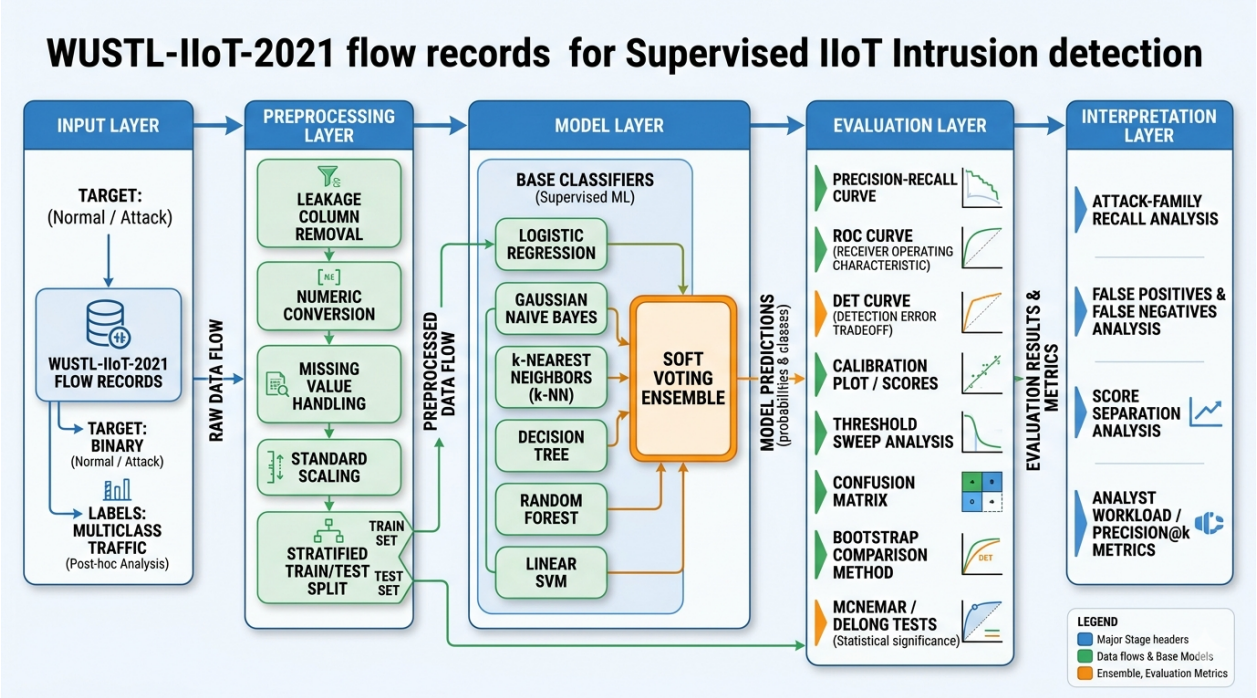


Figure 3.1: Modelling and evaluation pipeline. AI-generated illustration.

3.2 Supervised models and pipelines

Figure 3.1 follows the scripts: controlled inputs, split, fitting, score extraction and metric comparison. Scaling is fitted inside each pipeline using training data only Pedregosa et al., 2011; scikit-learn developers, 2024, which keeps linear, distance-based and SVM models comparable with the tree baselines. The figure also clarifies that family recall is calculated after prediction, not used as a training shortcut.

3.2.1 Algorithm design rationale

Model	Family	Strength	Known risk
Logistic Regression	Linear discriminative	Transparent baseline	Limited nonlinear fit.
Gaussian NB	Generative	Fast probability model	Independence mismatch.
k-NN	Instance based	Local discrimination	Slow prediction at scale.
Decision Tree	Tree based	Interpretable rules	Less stable than forests.
Random Forest	Ensemble trees	Nonlinear interactions	Higher training cost.
Linear SVM	Margin based	Strong linear margin	No native probabilities.
Soft Voting	Ensemble	Combines probabilities	Lower precision here.

Table 3.3: Algorithm design rationale.

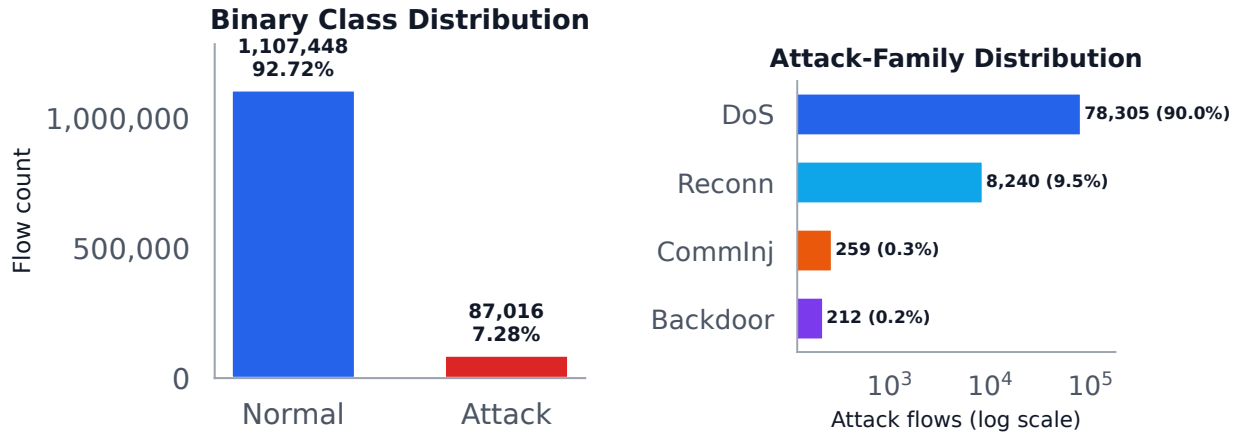
Table 3.3 sets up the rejection logic. Gaussian NB is later checked against correlation evidence, k-NN against prediction time, and Random Forest against both aggregate metrics and family-level misses.

4 Results

4.1 Univariate evidence

Univariate analysis shows the imbalance driving the report. Figure 4.1a has 1,107,448 normal flows (92.72%) and 87,016 attacks (7.28%). Figure 4.1b shows that DoS dominates the attack class with 78,305 flows (90.0%), while CommInj and Backdoor together contain fewer than 500 flows. This means aggregate attack metrics are not enough unless the rare families are checked separately.

4.1.1 Class and attack-family distributions



(a) Binary class distribution.

(b) Attack-family distribution.

Figure 4.1: Class imbalance and positive-class family skew.

Figure 4.1 gives two warnings: accuracy can be inflated by the 92.72% normal class, and attack recall can hide rare-family misses because DoS has about 302 times more rows than CommInj and 369 times more than Backdoor.

Category	Count	Interpretation
Normal	1,107,448	Majority class.
Attack	87,016	Minority class, 7.285%.
DoS	78,305	Dominant attack family.
Reconn	8,240	Secondary attack family.
CommInj	259	Rare attack family.
Backdoor	212	Rare attack family.

Table 4.1: Class and attack-family distribution.

Table 4.1 gives the same evidence as counts. The "Attack" row is the binary training target; the family rows explain why false negatives must later be checked by DoS, Reconn, CommInj and Backdoor. The table also shows that the modelling task is binary, while the operational interpretation is family-aware.

4.1.2 Data-quality summary

Check	Finding
Missing values	All 41 modelling features have zero missing values.
Feature ranges	Rate, load, byte, packet and timing variables show wide ranges.
Target skew	Attack prevalence is 7.285%, requiring imbalance-aware metrics.
Family skew	Backdoor and CommInj have low support relative to DoS.

Table 4.2: Univariate data-quality evidence.

Table 4.2 shows that missing values are not the issue. The practical problems are scale and imbalance: rate, load, byte, packet and timing variables differ greatly in range, and attacks are only

7.285% of flows.

4.2 Bivariate analysis

Bivariate analysis identifies feature groups that move with the target. Table 4.3 shows lower attack `dTtl`, higher `pLoss` and much higher rate/load values. Table 4.4 then shows that several engineered variables are near-duplicates. Together, these tables explain why a nonlinear model can use combined flow behaviour, while an independence-based model is risky.

4.2.1 Feature-target effects

Feature	Normal mean	Attack mean	Std. diff.
`dTtl`	62.68	2.37	3.236
`pLoss`	18.82	36.88	2.236
`SrcRate`	4,450.80	370,268.90	2.203
`Rate`	4,911.02	370,509.60	2.193
`SrcLoad`	2.52e6	1.84e8	2.172
`Load`	2.74e6	1.84e8	2.163

Table 4.3: Top bivariate feature-target effects by standardised difference.

Table 4.3 gives the strongest target shifts. `dTtl` falls from 62.68 for normal flows to 2.37 for attacks; `pLoss` rises from 18.82 to 36.88; and `SrcRate`/`Rate` rise from about 4–5 thousand to about 370 thousand. Attacks therefore change TTL, loss, rate and load together, so the later classifier should learn combinations rather than one threshold.

4.2.2 Highly correlated feature pairs

Feature A	Feature B	|r|	Implication
Min	RunTime	1.000	Duplicate timing structure.
SrcPkts	TotPkts	1.000	Packet-count dependence.
TcpRtt	SynAck	1.000	TCP timing dependence.
DstBytes	DAppBytes	1.000	Byte-count dependence.
SrcLoad	SrcRate	0.998	Rate/load coupling.
Load	Rate	0.998	Flow-rate coupling.

Table 4.4: Highly correlated feature pairs.

Table 4.4 shows dependence among engineered variables. Timing, packet and TCP pairs reach about 1.000 correlation, while SrcLoad–SrcRate and Load–Rate are about 0.998. This weakens the independence assumption behind Gaussian NB because the same flow signal is repeated through related variables.

4.3 Multivariate evidence

Multivariate analysis checks whether the bivariate shifts form separable structure. The PCA plot shows class geometry; the heatmap shows dependence among the same high-impact variables. These plots are not used as final classifiers, but they explain the shape of the learning problem before the model table is read.

4.3.1 PCA projection

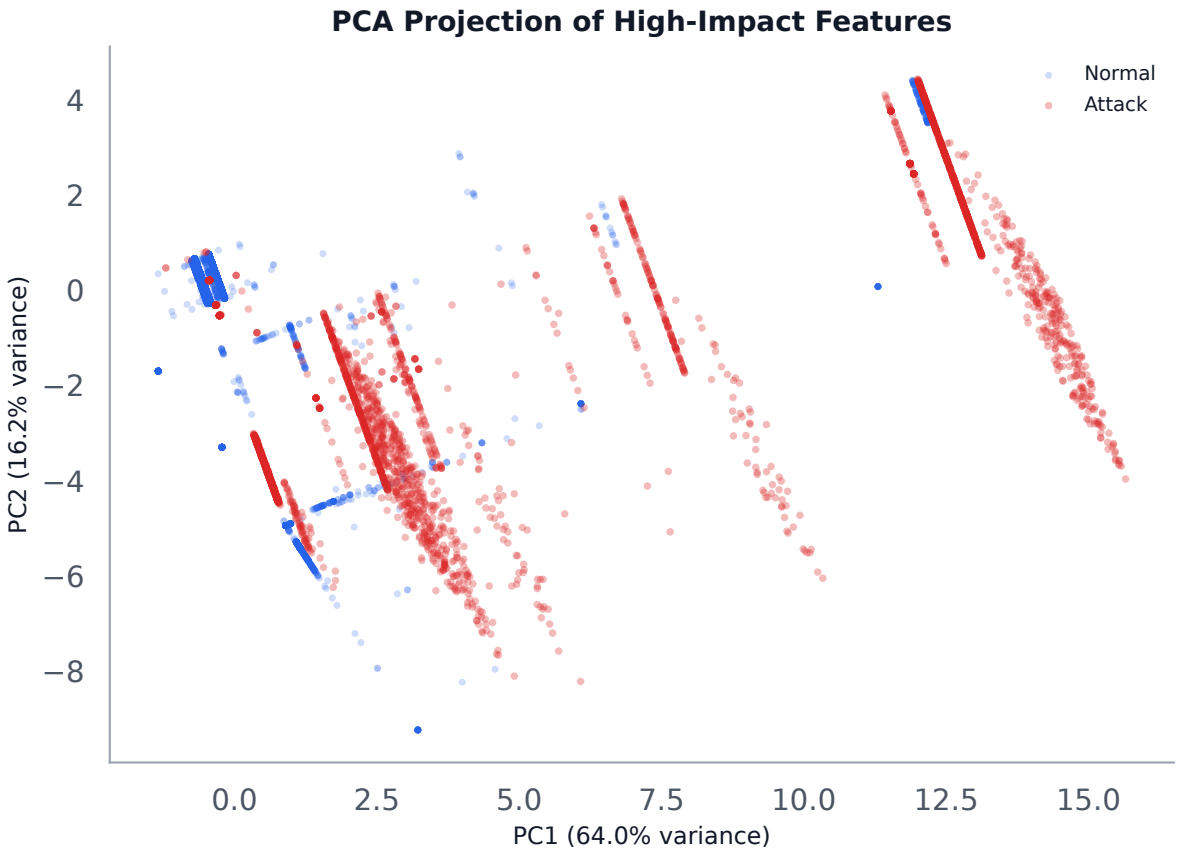


Figure 4.2: PCA projection of the training sample. The partial overlap supports multivariate supervised modelling rather than single-feature rules.

Figure 4.2 shows that PC1 and PC2 explain 64.0% and 16.2% of variance. Attack points form red diagonal bands, but normal points overlap them, so the data need supervised multivariate modelling rather than a simple visual threshold. The visible bands still support the later high model scores because they show structured separation, not random scatter.

4.3.2 Correlation heatmap

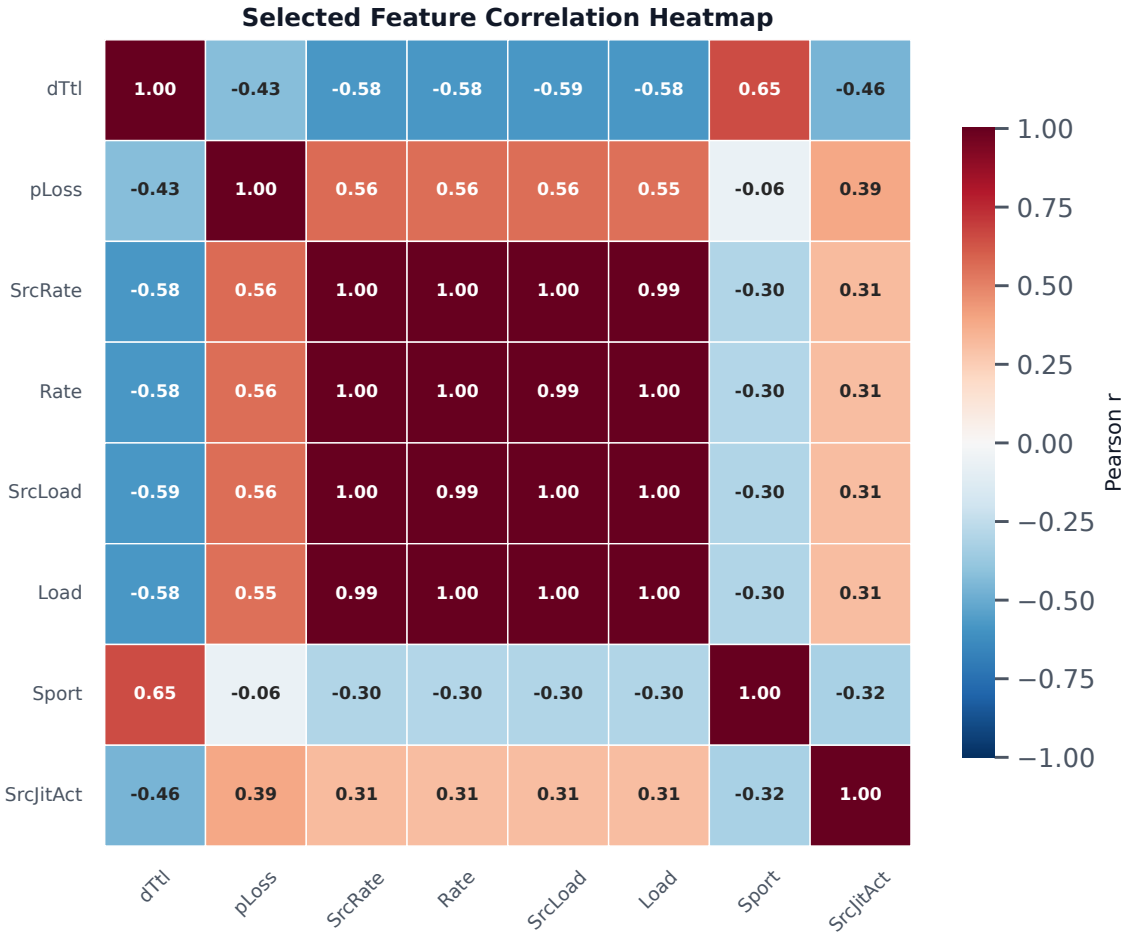


Figure 4.3: Correlation heatmap for selected high-impact variables. Strong dependence among flow features supports rejection of independence-based modelling assumptions.

Figure 4.3 makes the dependence explicit. `SrcRate`, `Rate`, `SrcLoad` and `Load` correlate from 0.99 to 1.00. `pLoss` is moderately positive with this block (0.55–0.56), while `dTtl` is negative with rate/load (-0.58 to -0.59) and positive with `Sport` (0.65). This is the heatmap evidence later used to reject Gaussian NB.

4.4 Comparative model performance

Table 4.5 reports the held-out comparison. Random Forest is strongest overall: accuracy 0.99994, precision 0.99943, recall 0.99971, F1 0.99957, MCC 0.99954 and PR-AUC 0.99999. Gaussian

NB has high recall (0.99810) but weak precision (0.86703) and PR-AUC (0.87752), while k-NN scores well but needs a runtime check. The table therefore separates statistical performance from deployability, which is assessed in the next section.

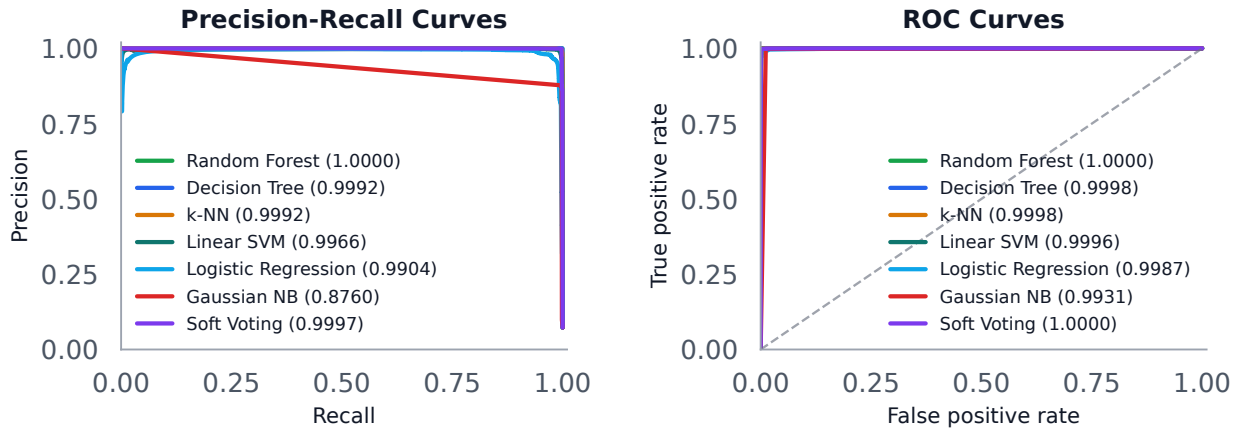
4.4.1 Held-out metric comparison

Model	Acc.	Prec.	Rec.	F1	MCC	ROC	PR
Random Forest	0.99994	0.99943	0.99971	0.99957	0.99954	1.00000	0.99999
Decision Tree	0.99987	0.99862	0.99966	0.99914	0.99907	0.99981	0.99922
k-NN	0.99939	0.99689	0.99477	0.99583	0.99550	0.99975	0.99918
Linear SVM	0.99557	0.94852	0.99311	0.97030	0.96821	0.99932	0.99384
Logistic Regression	0.98901	0.87485	0.99092	0.92927	0.92545	0.99860	0.98552
Gaussian NB	0.98871	0.86703	0.99810	0.92796	0.92456	0.99331	0.87752
Soft Voting	0.99311	0.91498	0.99810	0.95474	0.95207	0.99995	0.99962

Table 4.5: Main model comparison on the held-out test set.

Table 4.5 also shows why accuracy is misleading. Gaussian NB and Logistic Regression both exceed 0.988 accuracy, but their precision values (0.86703 and 0.87485) imply many more false alarms than Random Forest. Soft Voting has high recall and PR-AUC, but precision is only 0.91498. Random Forest is the only row that is near ceiling across precision, recall, F1, MCC and PR-AUC.

4.4.2 Precision–recall and ROC curves



(a) Precision-recall curves.

(b) ROC curves.

Figure 4.4: Model ranking curves. PR behaviour is more informative for minority-class intrusion detection.

Figure 4.4 confirms the ranking visually. In the PR panel, Random Forest, Soft Voting, Decision Tree and k-NN remain near the top boundary; Linear SVM is lower; Logistic Regression and Gaussian NB drop most. In the ROC panel, curves compress near the top-left because even Gaussian NB has ROC-AUC above 0.993, so PR behaviour is more useful here. This matches the table: ROC-AUC alone would make several weaker models appear closer to Random Forest than their precision behaviour supports.

4.4.3 Confusion matrix for the final model

		Predicted label: Normal	Predicted label: Attack
True label	Normal	**TN** 221,482	**FP** 8
True label	Attack	**FN** 5	**TP** 17,398

Figure 4.5: Final Random Forest confusion matrix: TN=221,482, FP=8, FN=5, TP=17,398.

Figure 4.5 converts the tuned Random Forest result into counts: 221,482 true negatives, 17,398 true positives, 8 false positives and 5 false negatives. The model makes only 13 errors in 238,893 test flows, missing 5 of 17,403 attacks. Its final PR-AUC is 0.999998, with precision 0.999540, recall 0.999713, F1 0.999627 and MCC 0.999597. The five missed attacks motivate the family-level diagnostic rather than a blanket claim of perfect detection.

4.5 Runtime and operational diagnostics

Runtime evidence matters because IIoT monitoring needs scalable prediction, not only strong offline scores. Table 4.6 and Figure 4.6 show that k-NN is fast to fit but slow to query, while Random Forest is slower to train but much faster at prediction. Training time is less important here because it happens offline; prediction time is more relevant to repeated monitoring.

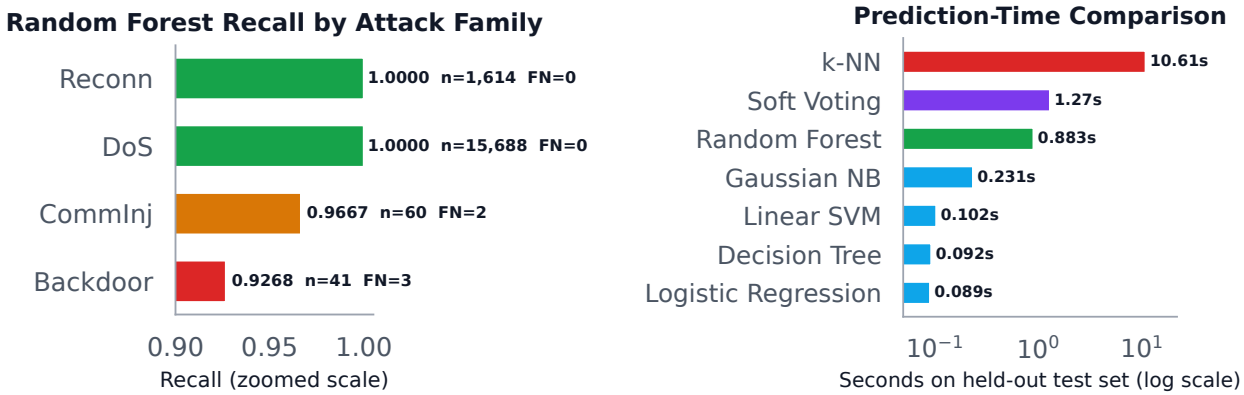
4.5.1 Training time, prediction time and footprint

Model	Train s	Predict s	Size MB	Proba	Suitability
Gaussian NB	0.300	0.231	0.004	Yes	Yes
Decision Tree	0.734	0.092	0.009	Yes	Yes
Random Forest	23.996	0.883	1.590	Yes	Yes
k-NN	0.074	10.610	25.638	Yes	Limited at scale
Linear SVM	139.637	0.102	0.003	No	Yes
Logistic Regression	159.318	0.089	0.003	Yes	Yes
Soft Voting	212.745	1.272	1.601	Yes	Yes

Table 4.6: Runtime and deployment suitability.

Table 4.6 answers the deployment-cost question. k-NN trains in 0.074s, but prediction takes 10.610s and the model occupies 25.638MB. Random Forest trains in 23.996s, predicts in 0.883s and uses 1.590MB, making it more practical at this scale. The comparison changes the interpretation of k-NN: its metric row is strong, but its deployment profile is weak.

4.5.2 Attack-family recall and prediction latency



(a) Recall by attack family.

(b) Prediction-time comparison.

Figure 4.6: Operational diagnostics for rare attack families and deployment cost.

Figure 4.6 combines the remaining risks. Random Forest detects all DoS and Reconn attacks in the test fold (15,688/15,688 and 1,614/1,614), but misses 2 of 60 CommInj and 3 of 41 Backdoor attacks. The latency panel shows k-NN at 10.61s, about twelve times slower than Random Forest at 0.883s. The final weakness is therefore narrow but important: rare attack families are less secure than the aggregate recall suggests.

4.6 Final model selection

Attack-family diagnostics add the final selection layer. Random Forest recall is perfect for DoS and Reconn, but falls to 0.9667 for CommInj and 0.9268 for Backdoor. All five false negatives come from those rare families, so the final recommendation includes rare-family monitoring. This preserves the strong Random Forest decision while avoiding an overclaim about uniform attack-family performance.

4.6.1 Decision matrix and summary

Model	Strength	Evidence and limitation	Decision
Random Forest	Strongest imbalance-aware profile	PR-AUC 0.999998 and 13 total RF errors; remaining misses are Backdoor/CommInj.	Selected
Gaussian NB	Fast probabilistic baseline	Correlated features violate independence assumptions; precision 0.86703 and PR-AUC 0.87752.	Rejected
k-NN	Strong local discrimination	PR-AUC 0.99918, but prediction time 10.610s and model size 25.638MB.	Rejected

Table 4.7: Final decision matrix.

Table 4.7 brings the evidence together. Random Forest is selected because it combines the best imbalance-aware profile, sub-second prediction on the held-out fold and only 13 total errors. Gaussian NB is rejected for correlated features and weak precision/PR-AUC. k-NN is rejected because its strong PR-AUC comes with slow prediction and a larger stored model. The selected model is therefore the best compromise between detection quality and operational cost.

5 Discussion and Conclusion

5.1 Synthesis of findings

The evidence supports Random Forest as the most defensible model. EDA shows 7.285% attack prevalence, DoS dominance and strong feature dependence. Model comparison then gives Random Forest the best imbalance-aware profile, and the confusion matrix reduces the final error to 8 false positives and 5 false negatives. The family plot completes the explanation by locating all five false negatives in CommInj and Backdoor.

5.2 Rejected models and deployment trade-offs

Gaussian NB is rejected because rate/load correlations reach 1.00 and several timing/packet fields are near-duplicates, conflicting with conditional independence. Its recall is high, but precision is 0.86703 and PR-AUC is 0.87752. k-NN is rejected because prediction takes 10.610s compared with 0.883s for Random Forest. These two rejections show different failure types: one statistical, one operational.

5.3 Limitations and final recommendation

The main limitation is external validity: one dataset and an offline random split may overstate performance under future traffic drift. Rare-family support is also limited, with only 41 held-out Backdoor and 60 CommInj examples, making those recall estimates less stable than DoS and Recon. In deployment, probability logs and family-level review would be needed to check whether the five-error pattern remains stable. The final recommendation is to use the tuned Random Forest as the coursework model, while validating it on time-based or external data and monitoring Backdoor/CommInj drift.

A Supplementary Reproducibility Note

A.1 Reproducibility materials

The supplementary project folder contains the dataset, experiment scripts, exported tables, generated figures and model artefacts used to produce this report. The final report body avoids raw file-path discussion, but the underlying work is reproducible from the supplied experiment scripts and generated outputs.

References

Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. New York: Springer.

Breiman, L. (2001). Random Forests. *Machine Learning* [online]. 45.1, pp. 5–32. DOI: 10.1023/A:1010933404324.

Chawla, N. V., Bowyer, K. W., Hall, L. O. and Kegelmeyer, W. P. (2002). SMOTE: Synthetic Minority Over-sampling Technique. *Journal of Artificial Intelligence Research* [online]. 16, pp. 321–357. DOI: 10.1613/jair.953.

Chen, W., Miao, C., Zhang, Z., Fung, C. S.-H., Wang, R., Chen, Y., Qian, Y., Cheng, L., Yip, K. Y., Tsui, S. K.-W. and Cao, Q. (2024). Commonly Used Software Tools Produce Conflicting and Overly-Optimistic AUPRC Values. *Genome Biology* [online]. 25, p. 118. DOI: 10.1186/s13059-024-03266-y.

Cortes, C. and Vapnik, V. (1995). Support-Vector Networks. *Machine Learning* [online]. 20.3, pp. 273–297. DOI: 10.1007/BF00994018.

Cruz Huayanay, A. de la, Bazán, J. L. and Russo, C. M. (2024). Performance of Evaluation Metrics for Classification in Imbalanced Data. *Computational Statistics* [online]. 40, pp. 1447–1473. DOI: 10.1007/s00180-024-01539-5.

Cullerne Bown, W. (2024). Sensitivity and Specificity versus Precision and Recall, and Related Dilemmas. *Journal of Classification* [online]. 41.2, pp. 402–426. DOI: 10.1007/s00357-024-09478-y.

Davis, J. and Goadrich, M. (2006). 'The Relationship Between Precision-Recall and ROC Curves'. In: *Proceedings of the 23rd International Conference on Machine Learning*, pp. 233–240. DOI: 10.1145/1143844.1143874.

Eid, A. M., Soudan, B., Nassif, A. B. and Injadat, M. (2024). Comparative Study of ML Models for IIoT Intrusion Detection: Impact of Data Preprocessing and Balancing. *Neural Computing and Applications* [online]. 36.13, pp. 6955–6972. DOI: 10.1007/s00521-024-09439-x.

Fawcett, T. (2006). An Introduction to ROC Analysis. *Pattern Recognition Letters* [online]. 27.8, pp. 861–874. DOI: 10.1016/j.patrec.2005.10.010.

Hastie, T., Tibshirani, R. and Friedman, J. (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. 2nd ed. New York: Springer.

Ismail, S., Dandan, S. and Qushou, A. (2025). Intrusion Detection in IoT and IIoT: Comparing Lightweight Machine Learning Techniques Using TON-IoT, WUSTL-IIOT-2021, and EdgeIIoTset Datasets. *IEEE Access* [online]. 13, pp. 73468–73485. DOI: 10.1109/ACCESS.2025.3554083.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M. and Duchesnay, É. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*. 12, pp. 2825–2830.

Powers, D. M. W. (2011). Evaluation: From Precision, Recall and F-Measure to ROC, Informedness, Markedness and Correlation. *Journal of Machine Learning Technologies*. 2.1, pp. 37–63.

Rathee, G., Iqbal, R., Kerrache, C. A. and Song, H. (2024). TrustNextGen: Security Aspects of Trustworthy Next Generation Industrial Internet of Things (IIoT). *IEEE Internet of Things Journal* [online]. 11.15, pp. 25568–25576. DOI: 10.1109/JIOT.2024.3361801.

scikit-learn developers (2024). *scikit-learn: Machine Learning in Python*. https://scikit-learn.org/stable/. Version 1.5.x, accessed 12 April 2026.

Sokolova, M. and Lapalme, G. (2009). A Systematic Analysis of Performance Measures for Classification Tasks. *Information Processing & Management* [online]. 45.4, pp. 427–437. DOI: 10.1016/j.ipm.2009.03.002.